HPC Matrix Multiplication Parallelization Guide

https://gemini.google.com/app/f3135692a529ca03

User prompt: Act as a High-Performance Computing (HPC) specialist. Explain how to effectively parallelize [insert algorithm/problem name, e.g., Matrix Multiplication / Parallel Sorting] using [OpenMP / MPI / CUDA]. Detail the decomposition strategy (data vs. task parallelism), data sharing attributes (shared vs. private variables), and potential race conditions or synchronization overheads to avoid.

Response: Assuming dense **Matrix Multiplication (**$C = A \times B$**)** using **OpenMP** (with parallel comparisons for **CUDA** and **MPI**), effective parallelization relies on **data parallelism** by partitioning the iteration space across execution threads.

Decomposition Strategy: Data vs. Task Parallelism

Matrix multiplication ($C_{i,j} = \sum_{k=0}^{N-1} A_{i,k} \cdot B_{k,j}$) is inherently suited for **data parallelism** because every element in output matrix C can be calculated independently without data dependencies across output coordinates.

- **Data Parallelism (Primary):** The iteration space (i, j) is decomposed across worker threads.
  - **1D Row Block Decomposition:** Matrix A and C are split by rows. Thread t computes a contiguous range of rows i. This is simple but limits scalability when thread count approaches matrix dimension N.
  - **2D Block Decomposition (Tiling/Blocking):** Matrices are partitioned into sub-blocks of size $T \times T$. Instead of operating on entire rows/columns, threads process sub-tiles that fit into L1/L2 cache (CPU) or Shared Memory (GPU). Tiling increases the operational intensity from memory-bound ($O(N^2)$ memory accesses) to compute-bound ($O(N^3)$ FLOPs).
- **Task Parallelism (Alternative):** Used primarily in recursive divide-and-conquer algorithms (e.g., Strassen's algorithm). Tasks dynamically partition sub-matrices onto an OpenMP task pool (`#pragma omp task`), but task allocation overhead makes this suboptimal for standard dense matrix multiplication.

Data Sharing Attributes & Memory Hierarchy Scoping

Proper variable scoping prevents data races and ensures optimal memory locality.

Paradigm	Data Level	Shared Memory / Global Variables	Private / Local Variables
OpenMP	CPU Shared Address Space	`shared(A, B, C, N)`: Matrices A, B, C are read-only or written to disjoint indices.	`private(i, j, k)` or local loop variables: Ensures thread-private loop counters and accumulation registers.
CUDA	GPU Massively Parallel	**Global Memory:** Array storage for A, B, C. **Shared Memory (`__shared__`):** Fast, block-local tile cache.	**Registers:** Thread-local scalar variables (i, j, accumulated sum register).
MPI	Distributed Memory	*None*. Memory is explicitly partitioned across isolated rank processes.	Rank-local sub-matrices sent via `MPI_Scatterv` / `MPI_Bcast` or 2D Torus shift (Cannon's Algorithm).

Race Conditions, Synchronization & Overhead Avoidance

1. Eliminating Race Conditions

- **Disjoint Writes:** Because each output element $C[i \cdot N + j]$ is computed by a single dedicated thread, no write-write race condition exists during the store phase.
- **Avoid Centralized Accumulation:** Never use a shared scalar for sums. Accumulate partial sums in a thread-private variable (or CPU register) before writing to $C[i \cdot N + j]$.

2. Mitigating Hardware & Software Overheads

- **False Sharing (CPU / OpenMP):**
  - *Problem:* Occurs when threads on separate CPU cores modify adjacent elements of C residing on the same 64-byte L1 cache line, causing constant cache invalidation bounce.
  - *Solution:* Align thread workloads along contiguous row boundaries (row-major order) or compute intermediate sums in private registers before writing out full row blocks.
- **Non-Contiguous Cache Misses (Stride Bottlenecks):**

- *Problem:* Accessing $B[k \cdot N + j]$ along column j causes large memory strides ($N \times 8$ bytes), missing the L1 cache.
- *Solution:* Reorder loops to $i \to k \to j$ or transpose B prior to execution so both matrix accesses are contiguous (stride-1).

- **GPU Memory Coalescing & Bank Conflicts (CUDA):**
  - *Problem:* Non-coalesced global memory reads drop GPU memory throughput drastically.
  - *Solution:* Threads within a 32-thread warp must access contiguous 32-bit/64-bit memory addresses. When using GPU `__shared__` memory tiling, pad tile dimensions (e.g., `[TILE_SIZE][TILE_SIZE + 1]`) to eliminate shared memory bank conflicts.
- **Synchronization Barriers:**
  - Avoid placing `#pragma omp barrier` or `#pragma omp atomic` inside inner loop nests.
  - In CUDA, use `__syncthreads()` strictly between sub-tile loading phases across thread blocks.

Concrete Parallel Implementations

OpenMP Implementation (Optimized C++ with Tiling)

```
#include <omp.h>

void matrix_multiply_omp_tiled(const double* __restrict A,
                               const double* __restrict B,
                               double* __restrict C,
                               int N, int TILE_SIZE) {
    // Explicit scoping prevents inadvertent variable sharing
    #pragma omp parallel for collapse(2) default(none) shared(A, B, C, N, TILE_SIZE) schedule(static)
    for (int ih = 0; ih < N; ih += TILE_SIZE) {
        for (int jh = 0; jh < N; jh += TILE_SIZE) {
            for (int kh = 0; kh < N; kh += TILE_SIZE) {

                // Micro-kernel operating within CPU L1/L2 cache
                for (int i = ih; i < ih + TILE_SIZE; ++i) {
                    for (int k = kh; k < kh + TILE_SIZE; ++k) {
                        double rA = A[i * N + k]; // Register load
                        #pragma omp simd // Auto-vectorize inner loop (AVX-512 / NEON)
                        for (int j = jh; j < jh + TILE_SIZE; ++j) {
                            C[i * N + j] += rA * B[k * N + j];
                        }
                    }
                }

            }
        }
    }
}
```

CUDA Kernel Implementation (Tiled Shared Memory)

```
#define TILE_SIZE 16

__global__ void matrix_multiply_cuda_tiled(const double* A, const double* B, double* C, int N) {
    __shared__ double sA[TILE_SIZE][TILE_SIZE];
    __shared__ double sB[TILE_SIZE][TILE_SIZE];

    int row = blockIdx.y * TILE_SIZE + threadIdx.y;
    int col = blockIdx.x * TILE_SIZE + threadIdx.x;
    double sum = 0.0; // Private thread register accumulator

    for (int phase = 0; phase < N / TILE_SIZE; ++phase) {
        // Coalesced global memory load into fast Shared Memory block
        sA[threadIdx.y][threadIdx.x] = A[row * N + (phase * TILE_SIZE + threadIdx.x)];
        sB[threadIdx.y][threadIdx.x] = B[(phase * TILE_SIZE + threadIdx.y) * N + col];

        __syncthreads(); // Barrier: Wait until tile is loaded by all threads in block

        for (int k = 0; k < TILE_SIZE; ++k) {
            sum += sA[threadIdx.y][k] * sB[k][threadIdx.x];
        }

        __syncthreads(); // Barrier: Prevent overwriting shared memory before work completes
    }

    if (row < N && col < N) {
        C[row * N + col] = sum; // Single global write, zero race condition
```

```
    }
}
```

User prompt: I am analyzing execution time data for a parallel algorithm tested across 1, 2, 4, 8, and 16 worker threads/nodes. The serial runtime is [X] seconds, and the parallel runtimes are [T1, T2, T4, T8, T16]. 1. Calculate the speedup and parallel efficiency for each thread count. 2. Analyze whether the speedup follows Amdahl's Law or Gustafson's Law. 3. Identify potential reasons for diminishing returns or parallel overhead at higher thread counts.

Response: 1. Speedup and Parallel Efficiency Formulas

To evaluate performance across $p \in \{1, 2, 4, 8, 16\}$ threads/nodes, use the following core metrics:

- **Speedup (S_p):** Measures the relative execution speed improvement of parallel execution over serial execution.

$$S(p) = \frac{T_{\text{serial}}}{T_p}$$

  (Note: If pure serial runtime T_{serial} is unavailable, relative speedup uses T_1 in the numerator: $S(p) = \frac{T_1}{T_p}$.)

- **Parallel Efficiency (E_p):** Measures the fraction of available processing power actively performing useful computation.

$$E(p) = \frac{S(p)}{p} = \frac{T_{\text{serial}}}{p \cdot T_p}$$

2. Speedup Model Analysis: Amdahl's vs. Gustafson's Law

- **Amdahl's Law (Strong Scaling — Fixed Problem Size):**
  - **Premise:** The total workload size remains constant as thread count p increases.
  - **Model:** Theoretical speedup is bounded by the strictly serial fraction s ($0 \le s \le 1$):

$$S(p) = \frac{1}{s + \frac{1-s}{p}}$$

  - **Behavior:** As $p \to \infty$, maximum speedup asymptotically approaches $\frac{1}{s}$.
  - **Classification:** If your benchmark evaluated a fixed dataset size across all thread counts, it follows **Amdahl's Law**. You can extract the effective serial fraction s from your data using:

$$s = \frac{\frac{1}{S(p)} - \frac{1}{p}}{1 - \frac{1}{p}}$$

- **Gustafson's Law (Weak Scaling — Scaled Problem Size):**
  - **Premise:** The workload expands proportionally with processor count p to keep total execution time roughly constant.
  - **Model:** Scaled Speedup grows linearly with p:

$$S(p) = p - s(p-1) = s + (1-s)p$$

  - **Classification:** If your benchmark scaled the problem size alongside thread count (e.g., N doubled when thread count doubled) while maintaining a flat execution time, it follows **Gustafson's Law**.

3. Potential Reasons for Parallel Overhead & Diminishing Returns

When parallel efficiency drops below 1.0 (100%) as thread count scales to 8 and 16, performance degrades due to specific system bottlenecks:

- **Memory Bandwidth Saturation (The Memory Wall):**
  - Multiple threads running on shared-memory sockets compete for the same memory bus and L3 cache.
  - Once memory bus bandwidth is fully saturated, additional worker threads spend cycles idling in memory stall states.
- **Cache Invalidation & False Sharing:**
  - When threads on separate physical CPU cores frequently write to independent variables that reside on the same 64-byte L1 cache line, constant cache coherence invalidation signals bounce across inter-core interconnects.
- **Synchronization & Lock Contention:**

- Global barriers (`#pragma omp barrier`, `MPI_Barrier`) force faster threads to wait for the slowest thread (straggler effect).
- Critical sections, atomic updates, or thread-safe memory allocations cause threads to serialize execution.

- **Communication Overhead:**
  - In distributed environments (MPI), message-passing latency and network bandwidth limits during collective operations (`MPI_Allreduce`, `MPI_Gather`) grow non-linearly with p.
  - In multi-threaded environments, thread creation, context-switching, and runtime scheduling overhead begin to dominate short execution loops.
- **Load Imbalance:**
  - Non-uniform workload distribution across threads leads to unequal compute times, reducing overall parallel efficiency.

If you reply with your numerical values for T_{serial} and $[T_1, T_2, T_4, T_8, T16]$, I will compute the exact speedup table, efficiency metrics, and estimated serial fraction s for your dataset. What are your specific execution time measurements?

User prompt: Explain how the computational demand and scaling laws of training Large Language Models (LLMs) on massive parallel GPU clusters impact energy consumption and carbon footprint. Contrast the raw throughput gains of high-performance parallel computing (HPC) with its environmental sustainability challenges.

Response: The compute requirement for pre-training Transformer-based Large Language Models (LLMs) is dictated by empirical scaling laws (e.g., Kaplan et al., Chinchilla). The total floating-point operations (C) required for a standard dense model pre-training run is approximated by:

$$C \approx 6 \cdot N \cdot D$$

where:

- N is the number of non-embedding model parameters.
- D is the total volume of training tokens.

As scaling laws demonstrate that test-loss L decreases as a power-law with compute ($L(C) \propto C^{-\alpha_C}$), modern models continue to scale both N and D. For example, pre-training a 400-billion parameter model on 15 trillion tokens requires approximately 3.6×10^{25} FLOPs.

Parallel Distributed Execution Mechanics

To complete training within acceptable wall-clock times (weeks to months), these FLOP requirements are distributed across tens of thousands of GPUs using multi-dimensional parallelism:

- **Tensor Parallelism (TP):** Splits individual matrix multiplications across intra-node GPUs over high-bandwidth NVLink connections.
- **Pipeline Parallelism (PP):** Distributes sequential layer blocks across separate GPU nodes.
- **Data Parallelism / ZeRO (FSDP):** Shards model states, gradients, and optimizer memory across worker ranks.
- **Context Parallelism (CP):** Partitions long sequence lengths along the self-attention dimension.

While parallelism delivers scalable compute throughput, it introduces significant communication barriers. Synchronization waits keep high-TDP (Thermal Design Power) processors idling at near-peak power levels, lowering total energy efficiency per effective FLOP.

Operational Energy, Carbon Footprint & Lifecycle Drivers

The environmental footprint of training an LLM on an HPC cluster is driven by operational electricity draw, regional grid intensity, and embodied hardware carbon.

Mathematical Carbon Accounting

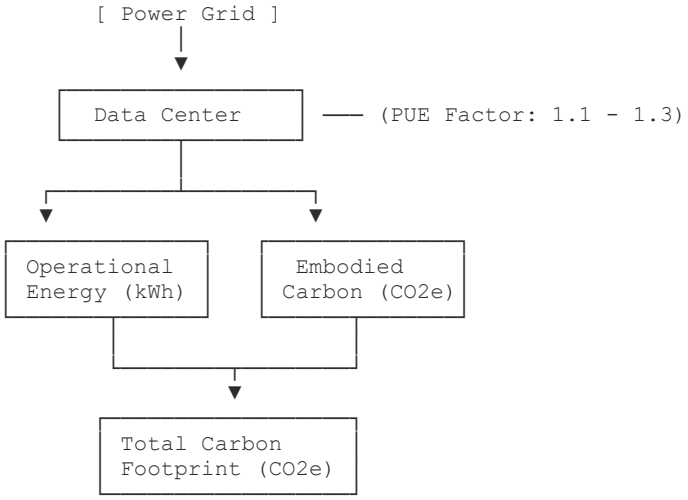
The total carbon emissions (CO_2e) generated by an LLM training run are modeled as:

$$\text{CO}_2\text{e}_{\text{total}} = \underbrace{\left(\frac{C}{\text{FLOPS}_{\text{effective}}} \cdot P_{\text{cluster}} \cdot \text{PUE} \right) \cdot I_{\text{grid}}}_{\text{Operational Carbon}} + \underbrace{\sum_{i=1}^{M} \left(\phi_{\text{GPU}} \cdot \frac{t_{\text{run}}}{t_{\text{lifespan}}} \right)}_{\text{Embodied Carbon}}$$

where:

- P_{cluster} is the active power draw of the compute nodes (in kW).

- PUE (Power Usage Effectiveness) represents data center infrastructure overhead (typically 1.1 to 1.3).
- I_{grid} is the carbon intensity of the local electrical grid ($\mathrm{gCO_2e/kWh}$).
- ϕ_{GPU} is the cradle-to-gate embodied manufacturing carbon per accelerator node.
- $t_{\text{run}}/t_{\text{lifespan}}$ is the fraction of total GPU useful lifespan consumed by the training job.



Key Sustainability Bottlenecks

1. **Gigawatt-Scale Power Demands:** Modern AI accelerator chips (e.g., NVIDIA H100/B200) operate at TDPs ranging from 700W to 1200W per socket. An 8-GPU server node consumes 10–12 kW, and a cluster scaling to 32,000+ GPUs demands 40–50+ MW of continuous power—equivalent to the consumption of a medium-sized city.
2. **Grid Intensity Variance:** Location heavily dictates operational carbon impact. Training a model on a coal-heavy grid ($I_{\text{grid}} \approx 600\text{–}800\ \mathrm{gCO_2/kWh}$) produces over 10 times the operational emissions compared to deployment on a clean hydroelectric or nuclear grid ($I_{\text{grid}} < 50\ \mathrm{gCO_2/kWh}$).
3. **Embodied Carbon & E-Waste:** Advanced semiconductor fabrication (sub-3nm process nodes, Silicon Interposers, HBM memory stacks) generates high manufacturing emissions. Rapid hardware replacement cycles (2–3 years) mean embodied carbon (ϕ_{GPU}) can constitute 30% to 50% of an accelerator's total lifecycle carbon footprint.
4. **Water Footprint:** High-density GPU racks generate extreme thermal loads ($> 100\ \mathrm{kW\ per\ rack}$), requiring liquid cooling or evaporative cooling towers that consume millions of liters of fresh water during multi-month training runs.

Throughput Optimization vs. Environmental Sustainability

HPC optimizations prioritize raw execution speed and throughput (minimizing time-to-solution), which often directly conflicts with energy efficiency and environmental goals.

Engineering Domain	HPC Throughput Priority	Environmental Sustainability Challenge
Hardware Utilization	Maximizing Model FLOPs Utilization (MFU) at high clock frequencies to achieve maximum TFLOPS/s.	GPUs operating at top dynamic voltage/frequency scaling (DVFS) levels run on the inefficient high end of the power curve, drawing disproportionately more energy for marginal speed gains.
Parallel Scaling	Scaling out to 100,000+ GPUs using reduced-precision tensor cores (FP8/FP4) to complete runs in minimum weeks.	Aggressive scale-out inflates total static cluster power draw, creates grid power-spike vulnerabilities, and accelerates thermal wear.
Network Interconnects	Uncapped NVLink and InfiniBand/RoCE bandwidth to minimize inter-node communication latency.	Continuous high-power transceiver draw across dense optical/copper fabrics adds persistent baseload energy overhead.
Fault Tolerance & Recovery	Synchronous, high-frequency parameter checkpointing to high-performance NVMe arrays to recover from node failures.	High-burst storage I/O and continuous background replication consume extra facility power while keeping compute pipelines stalled.
Facility Infrastructure	Redundant power delivery, high-rate liquid chilling, and high workload density.	Excessive direct water consumption, local utility grid congestion, and elevated PUE multipliers in warm ambient climates.

Sustainable Mitigations in Modern HPC

- **Carbon-Aware Scheduling:** Shifting non-time-critical training jobs dynamically based on real-time grid carbon intensity fluctuations and regional renewable availability.
- **Power Capping & DVFS Tuning:** Restricting maximum GPU TDP (e.g., limiting a 700W socket to 500W) often yields a 20% to 30% reduction in energy consumption with only a minor 5% to 8% throughput trade-off.
- **Algorithmic Efficiency:** Transitioning from dense architectures to sparse Mixture-of-Experts (MoE) models, reducing the active parameter count per token while maintaining total capacity.
- **Direct Liquid Cooling (DLC):** Implementing closed-loop direct-to-chip liquid cooling to lower data center PUE toward ~1.05 and virtually eliminate evaporative water loss.

What specific GPU cluster topology or scaling metric (e.g., MFU, PUE, or grid carbon intensity) would you like to analyze further?

Sources

1. How to estimate carbon footprint when training deep learning ... - PMC
2. Quantifying the Energy Consumption and Carbon Emissions of LLM

User prompt: How does the massive infrastructure required for modern parallel AI computing (e.g., thousands of interconnected GPUs) create a central concentration of power among tech giants? Analyze the ethical implications for academic research, smaller institutions, and global technology access equity.

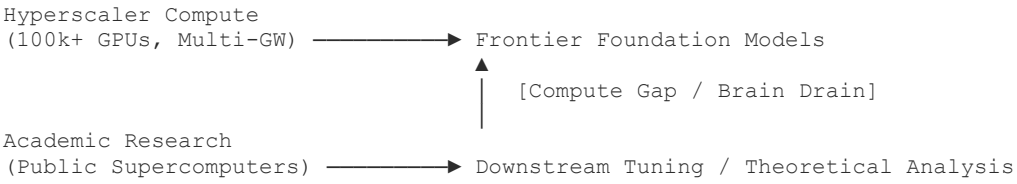
Response: The hardware footprint required to train and run state-of-the-art foundation models—clusters housing tens to hundreds of thousands of interconnected GPUs supported by multi-gigawatt power deliveries—has transformed AI computing from a software discipline into a capital-intensive infrastructure industry. This transition fundamentally alters the distribution of power across technology, academia, and global politics.

The Mechanics of Compute Concentration

The central concentration of power among a handful of tech giants (hyperscalers) is driven by three structural barriers to entry:

- **Extreme Capital Intensity:** Building and maintaining modern GPU clusters (e.g., 100,000+ accelerator pods) demands multi-billion-dollar capital expenditures. Beyond hardware purchase costs, the required support infrastructure—custom liquid cooling, high-density power substations, and ultra-low-latency optical fabrics—effectively limits frontier model development to multi-trillion-dollar corporations and state-backed entities.
- **Supply-Chain Monopsonies:** Hyperscalers command preferential allocation of advanced semiconductor nodes, High Bandwidth Memory (HBM), and specialized packaging. By pre-purchasing production capacity years in advance, tech giants restrict hardware access for non-corporate actors.
- **Vertical Integration Overhead:** Maximum compute throughput (Model FLOPs Utilization) requires proprietary, tightly integrated software stacks, orchestration software, and custom silicon (e.g., TPUs, internal ASICs). This engineering debt prevents smaller actors from simply assembling off-the-shelf hardware to compete on efficiency.

Ethical Implications for Academic Research



- **The Academic "Compute Gap":** Public research supercomputers are vastly outmatched by private corporate clusters. Even large public initiatives—such as the U.S. National AI Research Resource (NAIRR) or regional supercomputing consortia—provide researchers with compute allocations that are orders of magnitude smaller than those used in corporate labs.
- **Research Agenda Capture:** Because universities lack the hardware to train foundation models from scratch, academic agendas are forced to pivot toward downstream tasks: light fine-tuning, prompt engineering, or post-hoc interpretability. Core decisions regarding model design, pre-training data curation, and foundational alignment are dictated entirely by private incentives.
- **Loss of Independent Auditability:** When non-commercial researchers cannot replicate or directly inspect pre-training runs, scientific peer review loses its foundational role. AI safety, bias, and capabilities research become dependent on corporate transparency reports or restricted, rate-limited API access.
- **Academic Talent Drain:** The disparity in available compute, combined with severe compensation gaps, accelerates a "brain drain" of senior faculty and doctoral candidates from public universities into corporate research labs, eroding educational capacity for future generations.

Implications for Smaller Institutions and Startups

- **Compute Feudalism and API Lock-in:** Startups and non-profit organizations are increasingly relegated to acting as "wrapper" applications. Instead of innovating at the model layer, they rent inference time from closed-source corporate providers. This creates an economic rent extraction dynamic where value flows upward to cloud infrastructure owners.
- **Suppression of Public-Good Innovation:** Because commercial cloud computing is prohibitively expensive, early-stage research into high-impact, low-margin domains (such as rare disease diagnostics, low-resource language translation, or localized agricultural modeling) struggles to secure the necessary compute budget.

Global Technology Access Equity & Sovereign AI

- **The Global South Compute Divide:** The physical infrastructure of AI—data centers, subsea fiber routes, and chip fabs—is overwhelmingly concentrated in the Global North and East Asia. Developing nations risk falling into a state of *digital colonialism*, where regional data is extracted to train foreign proprietary models, which are then sold back to those regions as subscription services.
- **The "Sovereign AI" Dilemma:** To prevent total strategic dependency, nations are attempting to build "Sovereign AI" infrastructure. However, because the underlying hardware supply chains and orchestration software remain controlled by a few multinational providers, sovereign clouds often remain functionally dependent on foreign corporate ecosystems.
- **Environmental Externalities Displacement:** High-density data centers often place disproportionate stress on local communities by consuming large volumes of fresh cooling water and overburdening regional energy grids. The environmental burdens are localized near data facilities, while the intellectual property and economic profits accrue to distant corporate headquarters.

Would you like to analyze specific policy interventions—such as public compute trusts, mandatory hardware sharing mandates, or open-source hardware architectures—designed to democratize AI infrastructure?

Sources

1. Sovereign AI Infrastructure Market Share and Insights 2040

User prompt: Analyze how optimizations in parallel algorithms and model compression techniques (such as lower precision floating-point arithmetic or distributed pruning) might impact the fairness, accuracy, or bias of AI models deployed in societal decision-making.

Response: Optimizations designed to accelerate throughput and reduce memory overhead—such as quantization, pruning, and asynchronous parallel aggregation—frequently introduce **compression disparities**. While macro-level evaluation metrics (e.g., overall accuracy or mean squared error) often appear stable post-optimization, these efficiency gains are achieved by disproportionately degrading accuracy on underrepresented demographic slices, long-tail edge cases, and protected sub-populations.

Compression Disparities & "Pruning Bias"

- **Selective Subgroup Degradation:** Global pruning algorithms eliminate parameters based on weight magnitude or total loss reduction. Because rare demographic features contribute less to aggregate loss reduction during training, pruned sub-networks "cannibalize" capacity for underrepresented groups to preserve top-line performance.
- **Masked Macro Metrics:** In societal decision-making (e.g., automated resume screening or loan underwriting), a pruned model may maintain a 93% overall accuracy while its error rate on a 5% minority sub-population doubles, creating silent ethical and legal compliance failures.

Low-Precision Arithmetic & Dynamic Range Clipping

- **Numerical Dynamic Range Collapse:** Transitioning from FP32/FP16 to lower-precision formats (e.g., FP8, FP4, or INT8) significantly restricts exponent and mantissa dynamic ranges. Subtle, fine-grained decision boundaries required to evaluate complex social profiles get rounded off or clipped to zero.
- **Gradient Underflow in Rare Features:** During post-training quantization or quantized fine-tuning, low-amplitude feature activations associated with minority subgroups fall into uniform quantization bins. This erases nuance, forcing marginalized individuals into coarse majority-class classifications.

Parallel Asynchrony & Distributed Gradient Suppression

- **Asynchronous Update Staleness:** In large parallel GPU clusters, asynchronous parameter updates (e.g., bounded-staleness SGD) introduce noise. High-frequency majority gradient updates consistently overwrite the low-frequency gradient signals generated by rare training examples.
- **Lossy Communication Compression:** Distributed training frameworks often quantize or sparsify inter-node gradient transfers to eliminate network bottlenecks. Aggressive gradient compression drops small gradient components, suppressing learning dynamics for hard-to-classify demographic slices.

Societal Impact & Mitigation Frameworks

- **Fairness Disparity Amplification:** In high-stakes societal applications—such as recidivism risk scoring, credit approval, or healthcare resource allocation—compressed models violate core fairness criteria (e.g., equalized odds and demographic parity) far more frequently than their uncompressed counterparts.

- **Slicewise Audit Mandates:** Deploying optimized models safely requires shifting from aggregate validation datasets to mandatory **sliced evaluations** that benchmark false-positive and false-negative rates specifically across protected demographic intersections before and after compression.

For further background on how model compression affects machine learning bias, watch Sara Hooker on Fairness, Interpretability, and Model Compression, which details how pruning and quantization disproportionately impact underrepresented data features.

Would you like to explore specific algorithmic mitigation strategies, such as fairness-aware pruning or class-balanced quantization, to preserve sub-population accuracy during model compression?

Sources

1. Characterising Bias in Compressed Models by Sara Hooker
